ONLINE STORE · VEHICLE WORKFLOW GUIDE

Part 5

Putting it on the web: hosting, domain, upload, embed

A folder on your disk becomes a site that anyone can open. This part is the “render and deliver” stage: run it locally, pick a host, point a name at it, upload, and put the viewer next to the Buy button of your storefront provider. No servers to administer.

In this part

1. Running the site on your own machine first
2. What a static host is and which to pick
3. Your domain name and its sub-names
4. Uploading: the folder is the site
5. Putting the viewer next to the Buy button
6. Phones and touch
7. Speed: keeping the first visit under three seconds
8. Going-live checklist

Host and provider names are examples current as of writing; their free tiers and prices change. The mechanics (a static folder served over HTTPS, a DNS record pointing a name at it) do not. **check** marks items to confirm on the provider's own page.

1 · Running the site on your own machine first

Double-clicking `index.html` is not enough: browsers refuse to load module scripts and fetch files from a plain file path, so the stage stays blank. You need a tiny local web server. Python ships one, and you already have Python for the Blender tooling.

```
# in a terminal / PowerShell, inside the site folder
cd store-demo
python -m http.server 8765
# then open  http://localhost:8765/  in the browser.  Ctrl-C stops it.
```

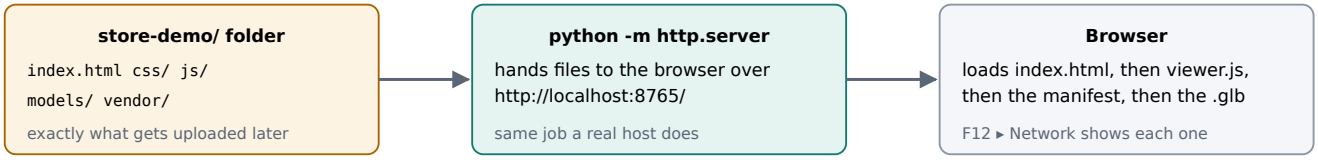


Figure 5.1 · Local first. The local server is the same contract as the real host: a folder, served. If it works here it works there.

> **The F12 panel is your Spreadsheet editor**
>
> Press F12 in the browser and open **Console** (error messages, the same role as Blender's system console) and **Network** (every file loaded, its size and time). Part 4 §12 lists the messages you will meet.

2 · What a static host is and which to pick

Your landing site has no server logic: it is files. A **static host** stores those files and serves them over HTTPS to the world. All of the ones below are free or nearly free at your traffic, take a folder upload or a Git push, give you HTTPS automatically, and let you attach your own domain name.

Host	How you upload	Cost at your scale	Notes
Cloudflare Pages	Drag the folder onto the dashboard, or connect a GitHub repo	Free	Fast worldwide; generous file limits (a 25 MB single-file cap is plenty for a .glb). Good default. **check**
Netlify	Drag-and-drop or Git	Free tier, bandwidth cap	Very simple; forms and redirects built in.
GitHub Pages	Git push to a repo	Free	This repository could host `store-demo/` directly; 1 GB site limit, soft 100 GB/month bandwidth.
Vercel	Git or CLI	Free for personal	Same idea; aimed at app frameworks you do not need.

Host	How you upload	Cost at your scale	Notes
Your storefront provider's pages	Their page editor	Included	Shopify / Payhip / etc. can host plain pages, but custom script support varies: the viewer may need to live on a static host and be embedded (§5).

> **Recommendation**
>
> Cloudflare Pages (or GitHub Pages, since you already keep the repository) for the landing site and the viewer, and the storefront provider for everything with money in it. The two are joined by links and one iframe, as Figure 5.3 shows.

3 · Your domain name and its sub-names

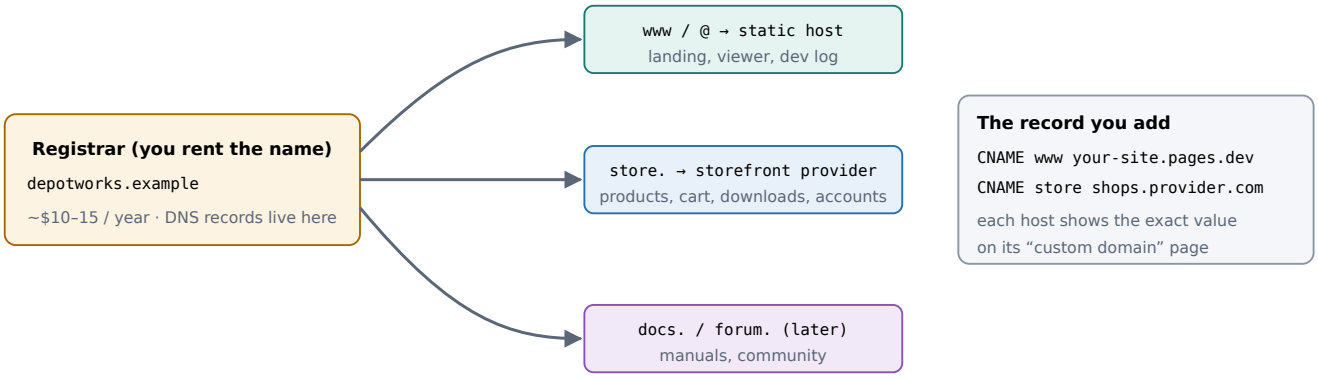


Figure 5.2 · One name, several front doors. A domain is like a collection: it can contain several sub-names, each pointed at a different host with one DNS record. Visitors never notice the join; `store.` simply looks like the next page.

1. **Buy the name** at any registrar (Cloudflare, Namecheap, Porkbun…). Pick the brand name from Part 2, not a railroad's name (Part 6 §6).
2. **On the static host**, open “custom domain”, enter `www.yourname`; it shows you a CNAME value. Add that record at the registrar. HTTPS is issued automatically within minutes to an hour.
3. **On the storefront provider**, do the same for `store.yourname`. Most providers also work fine on their own address if you skip this.
4. **Redirect the bare name** (`yourname` without www) to `www`; every host has a switch for it.

4 · Uploading: the folder is the site

Whatever you upload is served as-is, so the folder layout you tested locally is the layout online. Keep it exactly like the demo:

```
store-demo/            ← rename to  site/  or keep
├── index.html         landing page (the viewer lives here)
├── css/site.css
├── js/viewer.js
├── vendor/            three.js files, never edited
│   ├── three.module.js
│   └── jsm/  loaders/GLTFLoader.js  controls/OrbitControls.js
│             utils/BufferGeometryUtils.js
├── models/            one .glb + one .manifest.json per vehicle
├── img/               screenshots, logos (optional)
└── pages/             dev-log.html, about.html … (optional)
```

Method	Steps
Drag-and-drop	Host dashboard ▸ new project ▸ drop the folder. Repeat the drop for every update. Simplest; no tools.
Git push	Connect the host to this repository and tell it the folder is `store-demo`. Every `git push` redeploys in about a minute, and you keep history for free.
Command line	Each host has a one-line uploader (`wrangler pages deploy store-demo`, `netlify deploy`). Useful once you script the export (Part 7).

> **Never upload the product files to the static host**
>
> The .rwp / .zip installers people pay for go to the storefront provider's delivery system, which hides them behind a purchase. The static host is public: anything in the folder can be downloaded by anyone, including the web .glb, which is why that model is the low-poly web branch and not your game asset.

5 · Putting the viewer next to the Buy button

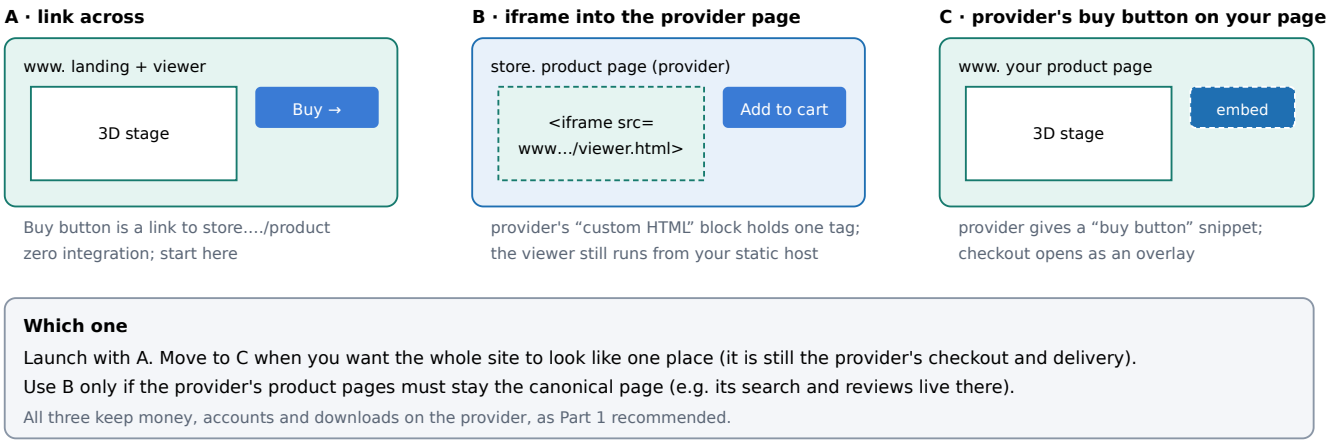


Figure 5.3 · Three ways to join the viewer and the checkout. None requires you to handle a payment.

A viewer-only page for embedding (option B)

Make a copy of `index.html` called `viewer.html` that keeps only the stage and the hotspot buttons, with the pool set to one manifest. Then in the provider's product editor, add a custom HTML block:

```
<iframe src="https://www.yourname/viewer.html?m=models/pcc_streetcar.manifest.json"
        width="100%" height="520" style="border:0" loading="lazy"
        title="Interactive 3D preview"></iframe>
```

To make `?m=` work, add one line near the top of `viewer.js`: read the parameter and, if present, use it as the pool.

```
const fromUrl = new URLSearchParams(location.search).get('m');
const POOL = fromUrl ? [fromUrl] : (window.MODEL_POOL ||
['models/sample_electric.manifest.json']);
```

6 · Phones and touch

- **Layout:** the demo's CSS stacks the stage above the panel under 900 px wide and hides the menu links; the stage keeps a 360 px height. Test by narrowing the browser window.
- **Touch:** OrbitControls already maps one finger to orbit and two to zoom/pan; a tap is a click (the six-pixel rule still applies). No change needed.
- **Hover labels** do not exist on touch; the button list is the touch user's way in, which is why both exist.
- **Budget harder:** phones are where the 60 k-triangle, 2048-px limits of Part 3 come from. The status line shows the triangle count; if a vehicle is heavy, give it a smaller web export for the pool and keep the big one for its product page.
- **Pixel ratio** is capped at 2 in the renderer so high-density screens do not render four times the pixels.

7 · Speed: keeping the first visit under three seconds

Lever	What to do
Model size	The biggest lever. 2–4 MB per .glb; textures JPEG where possible (Part 3). Draco compression halves mesh size but needs the decoder files added to `vendor/`; leave it for later.
Library size	`three.module.js` is about 1.2 MB uncompressed; hosts gzip it to ~300 kB automatically. Nothing to do.
Load order	The page text appears first; the viewer fetches after. Visitors read while the model arrives, and the status line says “Loading…”.
Lazy viewer	On pages where the stage is below the fold, start the viewer only when it scrolls into view (an `IntersectionObserver`, five lines). The landing page should load it immediately.

Lever	What to do
Caching	Static hosts send long cache headers for files that do not change. When you replace a .glb, give it a new name (`pcc_v2.glb`) or visitors may see the old one.
Images	Screenshots as JPEG ≤ 200 kB, width 1600 px; the hero stays 3D so there is no hero image at all.

8 · Going-live checklist

✓	Item
☐	Site runs on `localhost` with no console errors; every hotspot clicked
☐	Folder uploaded to the static host; the host's temporary address shows the same page
☐	Domain bought; `www` CNAME added; HTTPS padlock shows
☐	Storefront provider set up (Part 6); Buy links from the landing page open the right products
☐	Phone-width check: stage visible, buttons usable, no sideways scroll
☐	Network tab: total landing load under ~6 MB, model under budget
☐	NOTICE footnote and no trademarked names anywhere on the page (Part 6 §6)
☐	Dev-log page and a way to collect e-mail addresses exist, even if plain